

Retrieval-Augmented Generation and Information Retrieval

Retrieval-Augmented Generation and Information Retrieval

Retrieval-Augmented Generation, commonly called RAG, combines information retrieval with a language model. Instead of asking a language model to answer only from its pretraining, a RAG system first searches a collection of documents for relevant passages and then supplies those passages to the model as context. This approach can improve answers about private, domain-specific, or frequently changing information.

A basic RAG pipeline has several stages. During ingestion, documents are loaded and converted into clean text. The text is divided into chunks because embedding an entire long document as one vector can make retrieval too coarse. Each chunk is associated with metadata such as its source filename and page or section. The chunks are then converted into numerical embeddings and stored in a vector database.

Chunking is an important design choice. Fixed-size chunks are easy to implement and provide predictable sizes. Sentence-based chunking can preserve sentence boundaries. Paragraph or heading-based chunking can retain more semantic structure. Overlap between adjacent chunks helps prevent important information from being lost at boundaries. The best strategy depends on the document structure and the expected questions.

Embeddings represent text as vectors in a numerical space. Texts with related meaning should have vectors that are close according to a similarity measure. Cosine similarity is commonly used to compare normalized embedding vectors. An embedding model should be selected based on language support, quality, resource requirements, and licensing considerations.

A vector database or index stores embeddings and supports similarity search. Examples include FAISS, Chroma, Qdrant, and other vector stores. Persistence is important for an application that should not recompute embeddings every time it starts. A typical implementation has a separate indexing step and a query step. The indexing step creates or refreshes the stored vectors, while the query step loads the existing index.

At query time, the user's question is embedded using the same embedding model used for the document chunks. The system searches for the top-k most similar chunks. The value of k should be configurable because different questions may require different amounts of context. Retrieval results should retain metadata so that the final response can cite the source document and section or page.

The generation stage passes the retrieved context and the user question to a language model. A useful prompt should clearly instruct the model to answer from the supplied context and to avoid inventing unsupported facts. If the retrieved material does not contain the answer, the system should say that the information was not found rather than relying on unrelated model knowledge.

Grounding is a key concern in RAG. Retrieval quality depends on chunking, embedding quality, indexing, and query formulation. Generation quality depends on the selected model and prompt. A system can produce a fluent answer that is still incorrect if the retrieved context is irrelevant or incomplete. Evaluation should therefore test both retrieval and final-answer behavior.

Source citations improve transparency. Each retrieved chunk can be displayed with its filename and location metadata. A user can then inspect the source material behind an answer. Citation formatting should make it clear which documents were used. In a production system, citations should be generated from actual retrieval metadata rather than guessed by the model.

RAG systems also have limitations. Very small chunks can lose context, while very large chunks can reduce retrieval precision. Similar terminology across documents can produce ambiguous matches. Long contexts can increase latency and token usage. Documents with tables, images, headers, or complex layouts may require specialized extraction. Security is also important because retrieved documents can contain instructions that conflict with the application's system rules.

A practical RAG application should expose configuration such as chunk size, overlap, top-k, embedding model, vector-store location, and language-model choice. Logging can record which sources were retrieved without storing sensitive user questions unnecessarily. Evaluation should include answerable questions, cross-document questions, and questions that are deliberately outside the knowledge base. This last category tests whether the bot correctly refuses to invent an answer.